SOC ANALYST PORTFOLIO CASE STUDY

Detecting a Blocked Network Connection

UFW firewall telemetry, custom Wazuh rule engineering, and event investigation

RYAN BRYANT | SOC / CYBERSECURITY ANALYST CANDIDATE | AUGUST 2026

OUTCOME Configured host-firewall controls, generated a controlled TCP/8080 connection attempt, validated UFW evidence, engineered and tested Wazuh rule 100100, investigated ten Level 7 alerts, and confirmed approved Apache traffic remained available.

Figure 1. Home-lab architecture and firewall telemetry path.

Portfolio note: Authorized local simulation only. No external or production system was targeted. AI assistance is disclosed within this report.

1. Executive Summary

I tested Wazuh's ability to alert on an inbound connection blocked by the Ubuntu host firewall. I enabled UFW with a default incoming-deny policy, preserved approved SSH and Apache access, enabled medium logging, and added a documented deny rule for unused TCP port 8080.

A connection from the Wazuh server timed out, while Ubuntu recorded UFW BLOCK events showing source 10.10.10.3, destination 10.10.10.4, TCP SYN traffic, and destination port 8080. When no alert appeared in wazuh-alerts-*, I confirmed the syslog transport was established and treated the issue as a detection-content gap. I created and validated custom rule 100100. A repeated test produced ten Level 7 alerts, and a final HTTP 200 check proved approved Apache traffic remained available.

Project dimension	Implementation
Role performed	Detection tester, rule engineer, and Tier 1 incident analyst
Preventive control	Ubuntu UFW host firewall
Controlled traffic	10.10.10.3 to 10.10.10.4:8080/TCP
Preventive result	Connection timed out; no TCP session established
Detection result	Custom Wazuh rule 100100; Level 7; ten alerts
Availability result	Approved Apache TCP/80 returned HTTP 200
Disposition	True positive detection; expected authorized lab activity

2. Objectives

- Apply a least-access firewall policy without disrupting required services.
- Generate a safe blocked-connection event on an unused test port.
- Validate local endpoint evidence and syslog transport.
- Resolve a SIEM visibility gap with a tested custom rule.
- Investigate, scope, and accurately disposition the resulting alerts.

3. Lab Scope and Controls

Control	Implementation
Authorized systems	Wazuh and Ubuntu VMs in the personal home lab
Network	SOC-Lab NAT network 10.10.10.0/24
Preserved access	TCP/22 SSH and TCP/80 Apache explicitly allowed
Test port	Unused TCP/8080 explicitly denied
Telemetry	UFW/kernel through rsyslog TCP/514
Change safety	Backed up local_rules.xml and validated before restart

4. Implementation Workflow

01 BASELINE
Confirmed UFW was inactive.

02 ACCESS PRESERVATION
Allowed TCP/22 and TCP/80 before activation.

03 POLICY CONFIGURATION
Applied incoming deny, outgoing allow, medium logging, and enabled UFW.

04 CONTROLLED BLOCK
Denied TCP/8080 and verified the numbered rule.

05 TEST EXECUTION
Sent a connection from Wazuh and observed a five-second timeout.

06 ENDPOINT EVIDENCE
Confirmed UFW BLOCK records in /var/log/ufw.log.

07 TRANSPORT CHECK
Confirmed rsyslog maintained TCP/514 to Wazuh.

08 DETECTION ENGINEERING
Backed up local_rules.xml, added rule 100100, validated XML, and used wazuh-logtest.

09 ALERT INVESTIGATION
Restarted Wazuh, repeated the event, and investigated ten Level 7 alerts.

10 SERVICE ASSURANCE
Confirmed Apache still returned HTTP 200.

5. Key Commands

```
sudo ufw allow 22/tcp comment 'SOC Lab SSH'
sudo ufw allow 80/tcp comment 'SOC Lab Apache'
sudo ufw logging medium
sudo ufw enable
sudo ufw deny 8080/tcp comment 'SOC Lab Block Test'
curl -v --connect-timeout 5 http://10.10.10.4:8080/SOC-Lab-Blocked-Connection
```

SAFETY BOUNDARY The test targeted one unused port on an authorized VM. Required services were allowed before activation, and availability was verified afterward.

6. Evidence: Firewall Configuration

```
socstudent@inux-webserver01:~$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
socstudent@inux-webserver01:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
socstudent@inux-webserver01:~$ sudo ufw logging medium
Logging enabled
socstudent@inux-webserver01:~$ sudo ufw enable
Firewall is active and enabled on system startup
socstudent@inux-webserver01:~$ sudo ufw status verbose
Status: active
Logging: on (medium)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere                  # SOC Lab SSH
80/tcp                     ALLOW IN    Anywhere                  # SOC Lab Apache
22/tcp (v6)                ALLOW IN    Anywhere (v6)             # SOC Lab SSH
80/tcp (v6)                ALLOW IN    Anywhere (v6)             # SOC Lab Apache

socstudent@inux-webserver01:~$
```

Figure 2. Active UFW policy with required service exceptions.

```
socstudent@inux-webserver01:~$ sudo ufw deny 8080/tcp comment 'SOC Lab Block Test'
Rule added
Rule added (v6)
socstudent@inux-webserver01:~$ sudo ufw status numbered
Status: active

     To                         Action      From
     --                         ------      ----
[ 1] 22/tcp                     ALLOW IN    Anywhere                  # SOC Lab SSH
[ 2] 80/tcp                     ALLOW IN    Anywhere                  # SOC Lab Apache
[ 3] 8080/tcp                   DENY IN     Anywhere                  # SOC Lab Block Test
[ 4] 22/tcp (v6)                ALLOW IN    Anywhere (v6)             # SOC Lab SSH
[ 5] 80/tcp (v6)                ALLOW IN    Anywhere (v6)             # SOC Lab Apache
[ 6] 8080/tcp (v6)              DENY IN     Anywhere (v6)             # SOC Lab Block Test

socstudent@inux-webserver01:~$
```

Figure 3. Explicit TCP/8080 deny rule.

7. Evidence: Blocked Request and Endpoint Log

Figure 4. Controlled connection blocked by UFW.

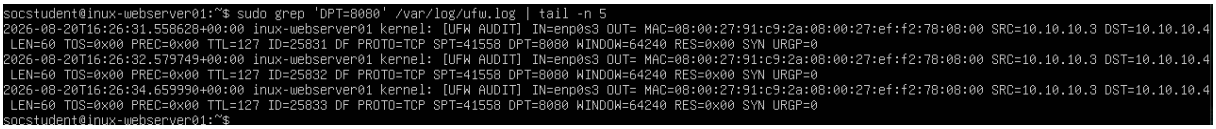

Figure 5. Local UFW audit and block evidence.

Field	Observed value
Action	UFW BLOCK
Source / destination	10.10.10.3 / 10.10.10.4
Protocol / flag	TCP / SYN
Destination port	8080
Result	Timed out; no session established

8. Detection Engineering and Rule Validation

The endpoint record existed and rsyslog maintained an established TCP connection to Wazuh, but the default rules produced no indexed alert. I therefore implemented a narrowly scoped local rule using a reserved custom-rule ID. I backed up the original file, detected and recovered from malformed XML before service restart, validated the corrected XML, and tested the rule against a representative UFW record.

```
<rule id="100100" level="7">
  <match>UFW BLOCK</match>
  <description>UFW blocked an inbound network connection.</description>
  <group>firewall,network_connection_denied,</group>
</rule>
```

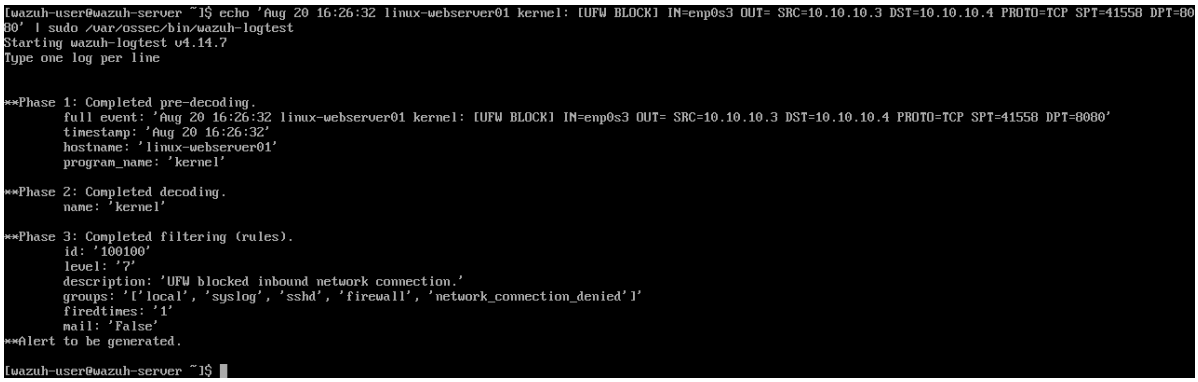

Figure 6. Successful custom-rule test before deployment.

ENGINEERING JUDGMENT A backup and predeployment validation prevented malformed XML from interrupting the Wazuh manager.

9. Evidence: Wazuh Event Investigation

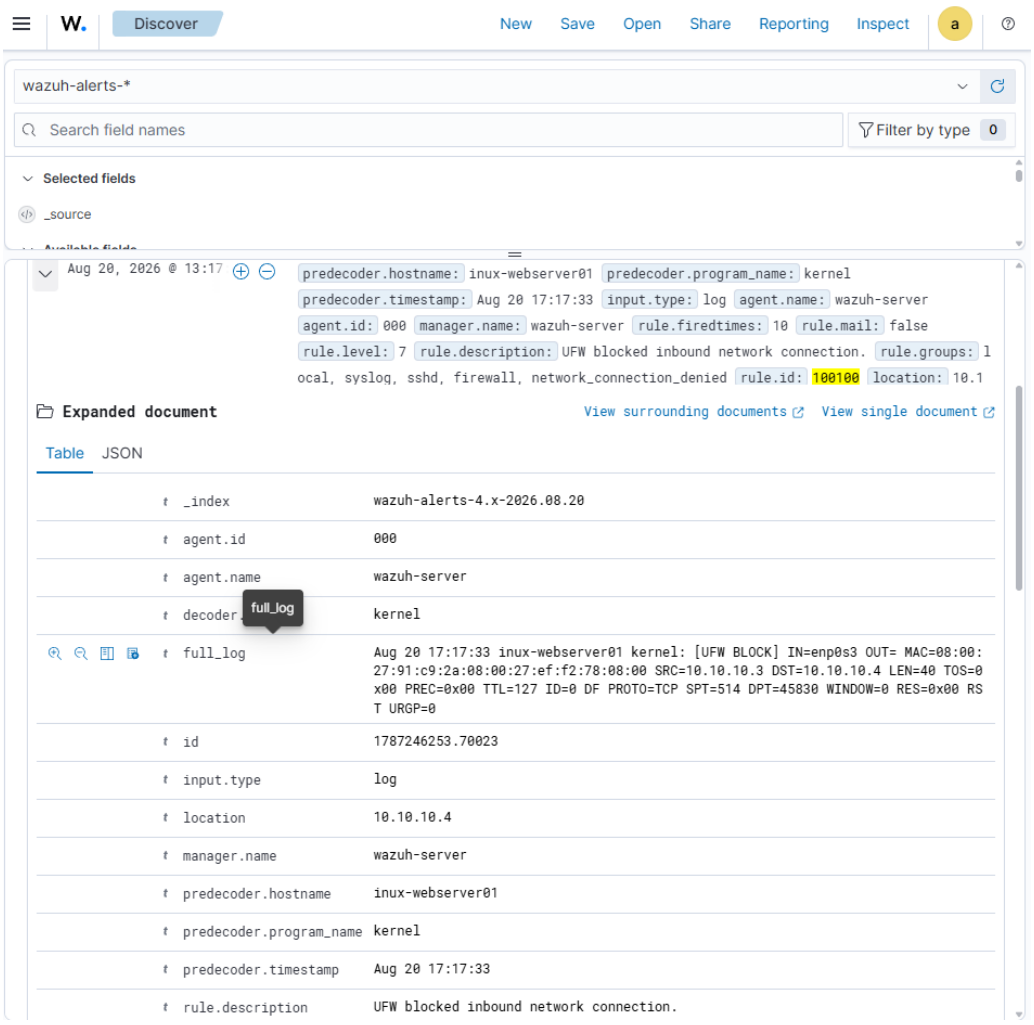

Figure 7. Expanded Wazuh firewall alert.

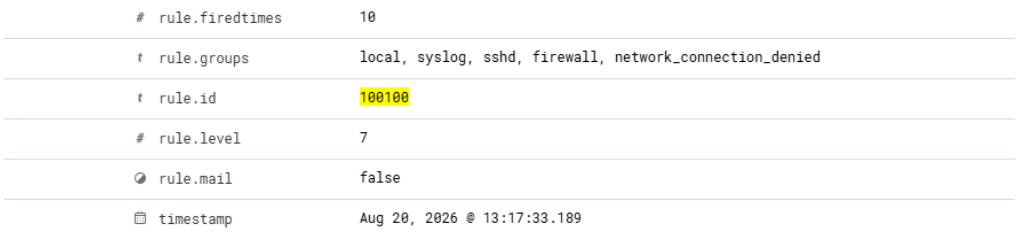

Figure 8. Rule metadata and cumulative match count.

Alert field	Observed value
Rule	100100 — UFW blocked an inbound network connection
Level / fired times	7 / 10
Program / decoder	kernel / kernel
Location	10.10.10.4
Groups	firewall, network_connection_denied
Manager	wazuh-server

10. Triage and Final Disposition

Question	Assessment
What happened?	A controlled client attempted TCP/8080 and UFW dropped the SYN packets.
Was access successful?	No. curl timed out and no TCP session was established.
Was the source identified?	Yes. 10.10.10.3, the authorized Wazuh lab server.
Why ten alerts?	Connection retries generated multiple blocked packets; firedtimes reached 10.
Was approved service affected?	No. Apache on TCP/80 returned HTTP 200.
Classification	True positive detection; expected benign activity in an authorized lab.

FINAL DISPOSITION TRUE POSITIVE — EXPECTED LAB ACTIVITY. The preventive control blocked the connection, the SIEM detected the activity, and approved service availability was preserved.

11. Availability Verification

Figure 9. Approved Apache service remained available.

12. Recommended Response in a Real Environment

- Validate source, destination, port, protocol, firewall action, and asset ownership.
- Determine whether attempts are isolated, repeated, distributed, or part of a broader scan.
- Correlate firewall, authentication, application, endpoint, DNS, and network telemetry.
- Verify denied traffic did not shift to another port or approved service.
- Block or rate-limit confirmed hostile sources according to policy.
- Escalate when attempts span multiple ports or hosts, become persistent, or precede successful access.

13. MITRE ATT&CK and Analyst Judgment

A single blocked connection does not prove scanning or exploitation. MITRE ATT&CK T1046 (Network Service Discovery) may become relevant when attempts cover multiple ports or hosts, but this lab observed one controlled connection to one port. The report therefore describes the evidence without assigning unsupported attacker intent.

14. Assistance and Source Transparency

This project was completed by Ryan Bryant in his personal home lab with assistance from OpenAI ChatGPT (“Sarah”). ChatGPT helped plan the safe workflow, troubleshoot log visibility, locate official documentation, and edit this portfolio narrative. Ryan personally operated the VMs, entered commands, configured UFW, modified and validated the Wazuh rule, generated the event, investigated the alerts, captured screenshots, and confirmed the findings. ChatGPT did not independently access or operate the lab computer.

15. Skills Demonstrated

- Linux UFW configuration
- Least-access firewall policy
- Syslog transport validation
- Endpoint-versus-SIEM evidence comparison
- Wazuh custom-rule engineering
- XML validation and safe rollback
- wazuh-logtest use
- SIEM investigation and event correlation
- Service-availability validation
- Incident triage and disposition

16. Employer-Facing Project Summary

RESUME BULLET Built and validated a Wazuh firewall detection workflow in an isolated Ubuntu lab: configured UFW controls, verified TCP/514 telemetry, engineered custom Level 7 rule 100100, investigated ten blocked TCP/8080 events, and confirmed approved Apache availability.

Interview talking points

- How I separated a telemetry problem from a rule-coverage gap.
- Why I backed up and validated XML before restarting the SIEM.
- How endpoint logs proved prevention before an alert existed.
- Why ten packet alerts did not mean ten successful connections.
- How I verified security controls without disrupting an approved service.

17. References and Validation Sources

Wazuh custom rules
Wazuh wazuh-logtest
Wazuh rule testing
Wazuh rules syntax
Ubuntu UFW manual
MITRE ATT&CK T1046
OpenAI key concepts